FIELD PROJECT REPORT

(Course Code: CA-310 | Field Project | FP-2)

on

SpareGo

Spare Parts Order and Delivery Management System for Local Auto-Spare-Parts Shops

Submitted by

[Student name(s) and roll number(s)]

Under the guidance of

[Guide name]

[Class, Department, College, University, Academic year]

CERTIFICATE (College / Mentor)

This is to certify that the Field Project "SpareGo – Spare Parts Order and Delivery Management System" was carried out by [names] during the academic year [year] under my guidance.

Guide: _____ HOD: _____ Principal: _____

ORGANIZATION CERTIFICATE

[Paste the signed certificate from the spare parts shop where the field study was done: shop name, address, owner signature and stamp.]

ACKNOWLEDGEMENT

We thank our guide [name], the owner of [shop name] for allowing the study and sharing honest feedback, our HOD, faculty and family.

CONTENTS

Chapter 1: Introduction

Chapter 2: Problem Identification and Feasibility Study

Chapter 3: Requirement Analysis

Chapter 4: System Design

Chapter 5: Implementation

Chapter 6: Conclusion and Future Scope

Appendix A: Questionnaire | Appendix B: Test Cases | References

Chapter 1: Introduction

1.1 Background of the Project

Local auto-spare-parts shops serve mechanics, garages and vehicle owners. Customers usually phone or visit the shop, ask for a part by vehicle model, and the shop sends it by a delivery boy. Orders are noted on paper or WhatsApp, availability is checked by searching shelves, and customers repeatedly call to ask "where is my part?". Wrong parts, lost orders and wasted time are common, and a mechanic waiting for a part loses working hours.

SpareGo is a web application that digitises this workflow: parts catalogue searchable by vehicle model, stock tracking with low-stock alerts, order placement with automatic invoice, packing and rider assignment, a public tracking page for customers, and OTP-verified delivery.

1.2 Objectives of the Project

- To study the existing manual order and delivery process of a spare parts shop through field visit, interviews, observation and questionnaires.
- To identify the problems and define functional and non-functional requirements.
- To design the system using DFD, ERD, use case, flowchart and database schema.
- To develop a working application using Python, Flask and SQLite.
- To add smart features: search by vehicle model, low-stock alerts, order tracking link on WhatsApp, delivery OTP and rider performance report.
- To test the system manually and compare the existing and proposed systems.

1.3 Scope

Single shop, one owner/staff login, in-house riders. Online payment, GPS tracking and multi-shop support are future scope.

Chapter 2: Problem Identification and Feasibility Study

2.1 Description of the Current System

The field study was done at [shop name, address] on [dates]. Owner: [name]; years in business: [years]; number of delivery boys: [n]; approximate orders per day: [n]. The current process is:

- Customer phones or visits and asks for a part by vehicle and part name.
- Owner or helper searches shelves/memory; availability is told verbally.
- Order is written in a notebook or WhatsApp chat; bill is hand-written.
- A delivery boy is told verbally; the customer calls to ask about delivery status.
- Payment collection and delivery proof are not recorded.

[Write 1–2 paragraphs in your own words about what you actually observed.]

2.2 Limitations of the Existing System

No.	Problem	Impact
1	No searchable catalogue by vehicle model	Slow response, wrong part given
2	Stock checked by memory	Out-of-stock found late; lost sales
3	Orders on paper / WhatsApp	Lost or forgotten orders
4	No order status for customer	Repeated calls to shop
5	No delivery proof	Disputes about delivery and payment
6	No rider or sales reports	Owner cannot measure performance

[Confirm/replace this table with your real survey findings.]

2.3 Feasibility Study

Type	Assessment
Technical	Free tools (Python, Flask, SQLite); runs on any PC; customer needs only a phone browser. Feasible.
Economic	No licence cost. Feasible.
Operational	Few screens, simple steps; owner confirmed he can use it [[confirm in interview]]. Feasible.
Schedule	Completed within the 60-hour project period. Feasible.

Chapter 3: Requirement Analysis

3.1 Method of Data Collection

Primary data was collected through field visit, interview, questionnaire and observation. The questionnaire is in Appendix A.

3.2 Field Visit Details

Visit	Date	Place	Purpose
1	[[date]]	[[shop]]	Introduce project, observe order and delivery process
2	[[date]]	[[shop]]	Interview owner, collect questionnaires
3	[[date]]	[[shop]]	Demonstrate prototype and take feedback

3.3 Interview Details

Interviewee: [owner name]. Duration: [minutes]. Main points: [real answers about order volume, delivery delays, wrong-part returns, how customers ask for status].

3.4 Questionnaire / Survey Method

A 10-question questionnaire was given to [N] respondents ([shop owners, mechanics, customers]).

3.5 Observation Details

We observed the shop for [hours] on [days] and noted how orders were taken, how long search took and how delivery was handled. [Write real observations.]

3.6 Summary of Collected Data

Question	Result (fill from your survey)
Customers who call to ask delivery status	[[x of N]]
Orders delivered late / wrong part sent	[[x of N]]
Orders recorded on paper/WhatsApp only	[[x of N]]
Would like order tracking link	[[x of N]]
Willing to use an app for orders	[[x of N]]

NOTE: Numbers must come from the real survey; do not invent them.

3.7 Functional Requirements

- FR1: Secure shop login.

- FR2: Add parts with vehicle model, price, stock; search by part/brand/vehicle; restock.
- FR3: Add customers (individual/mechanic/garage) and riders.
- FR4: Place order with stock check; stock reduces automatically; delivery fee rule (free above Rs 1000).
- FR5: Order life cycle: Placed → Packed → Out for Delivery → Delivered.
- FR6: Rider must be assigned before dispatch.
- FR7: Delivery completes only with correct customer OTP.
- FR8: Public tracking page (order number + phone) and WhatsApp tracking link.
- FR9: Printable invoice.
- FR10: Dashboard, low-stock alerts, most-ordered parts, rider performance and 7-day order value.

3.8 Non-Functional Requirements

- Usability: simple pages; tracking page is mobile friendly.
- Performance: page loads in under 1 second for small data.
- Reliability: foreign keys and CHECK constraints; invalid input rejected.
- Security: hashed passwords, login protection, OTP for delivery, tracking needs phone match.
- Portability: runs on any OS with Python.

Chapter 4: System Design

4.1 Use Case Diagram

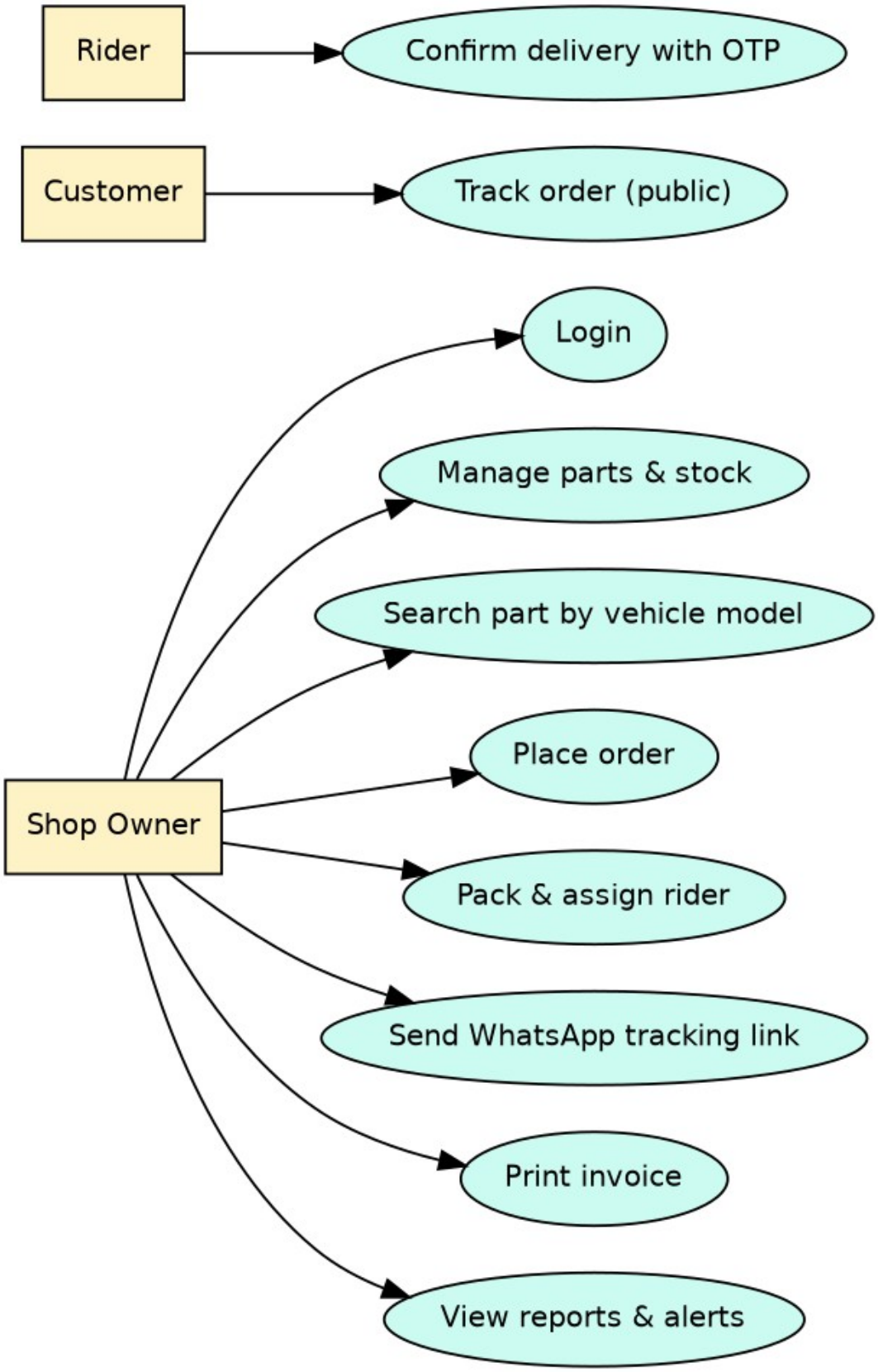

Rider
Confirm delivery with OTP
Customer
Track order (public)
Login
Manage parts & stock
Search part by vehicle model
Place order
Shop Owner
Pack & assign rider
Send WhatsApp tracking link
Print invoice
View reports & alerts

Figure 4.1: Use case diagram

4.2 Flowchart

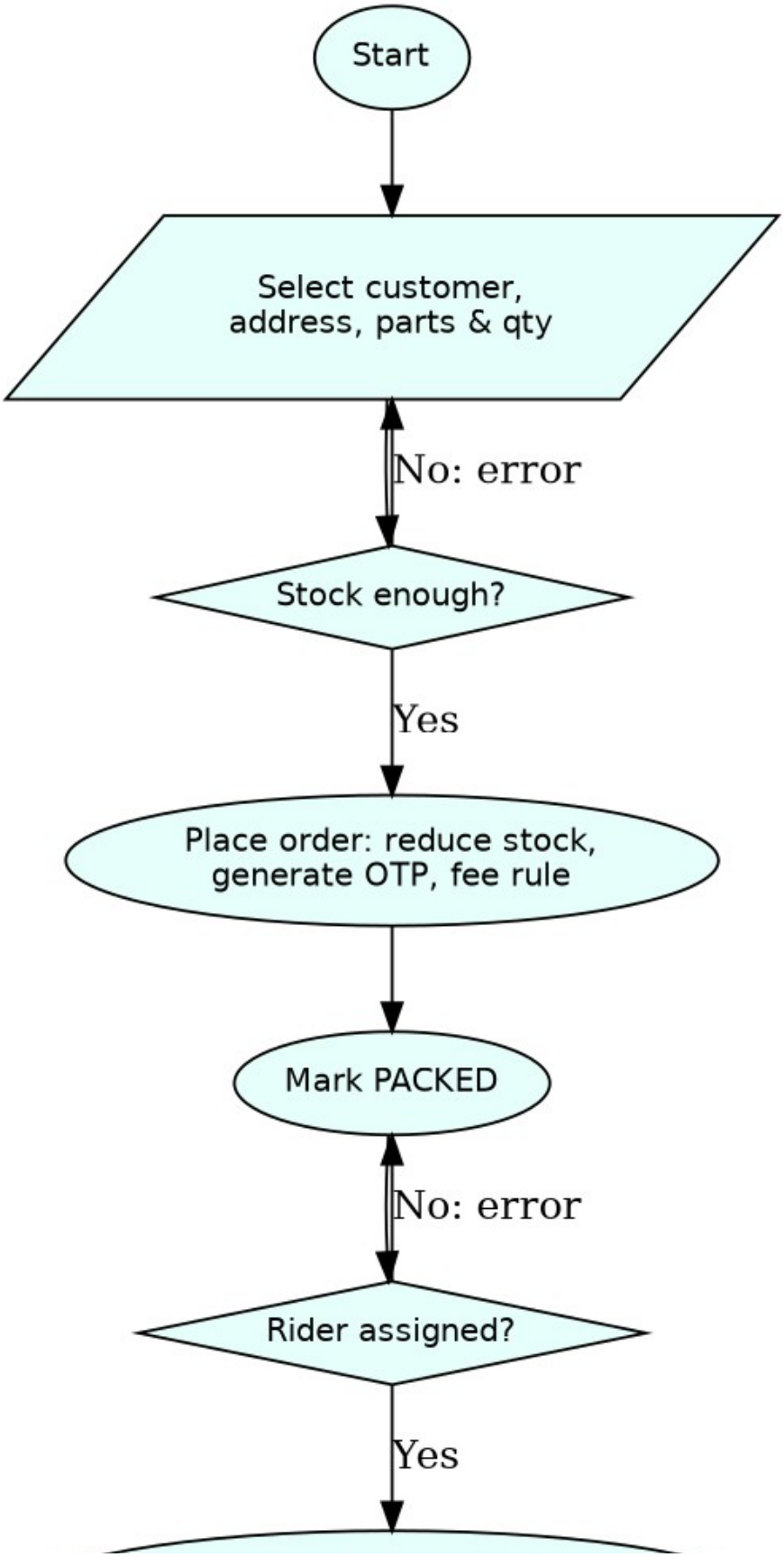

Start
Select customer, address, parts & qty
No: error
Stock enough?
Yes
Place order: reduce stock, generate OTP, fee rule
Mark PACKED
No: error
Rider assigned?
Yes

Figure 4.2: Order and delivery flowchart

4.3 Data Flow Diagrams

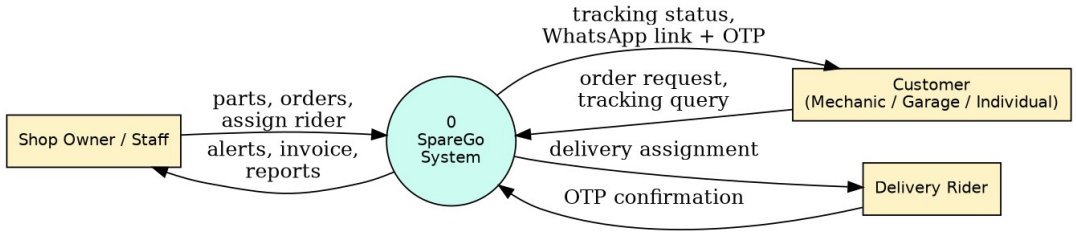


Figure 4.3: DFD Level 0

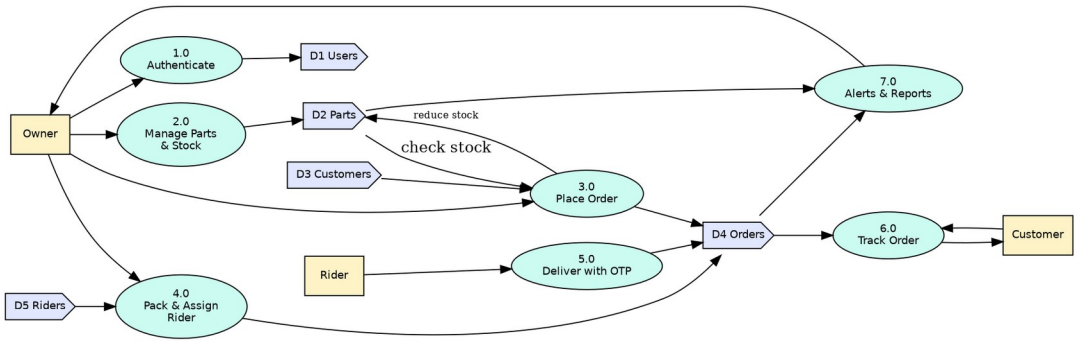


Figure 4.4: DFD Level 1

4.4 Entity Relationship Diagram

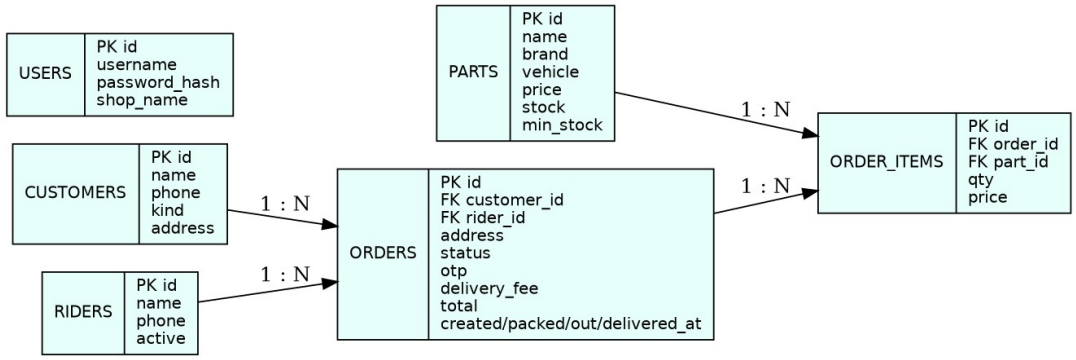


Figure 4.5: ER diagram

4.5 Database Design

Table	Columns	Keys / constraints
users	id, username, password_hash, shop_name	PK id; username UNIQUE
parts	id, name, brand, vehicle, price, stock, min_stock	PK id; price>=0, stock>=0
customers	id, name, phone, kind, address	PK id
riders	id, name, phone, active	PK id

orders	id, customer_id, rider_id, address, status, otp, delivery_fee, total, created/packed/out/delivered_at	PK id; FK customer_id, rider_id
order_items	id, order_id, part_id, qty, price	PK id; FK order_id, part_id

Tables are in third normal form. The unit price is copied into order_items so old invoices do not change when part prices change.

4.6 Business Rules

- Delivery fee Rs 40; free if subtotal is Rs 1000 or more.
- Order cannot be placed if quantity exceeds stock.
- Status can only move forward one step at a time.
- Rider is mandatory before Out for Delivery; correct OTP is mandatory for Delivered.
- Low-stock alert when stock is less than or equal to minimum stock.

Chapter 5: Implementation

5.1 Technologies Used

Component	Technology
Language	Python 3
Framework	Flask with Jinja2
Database	SQLite
Front end	HTML5, CSS3 (offline)
Security	Werkzeug hashing, Flask sessions
Tools	VS Code, Git/GitHub, Playwright, Graphviz

5.2 Description of Modules

Module	Description
Authentication	Login/logout, session protection
Parts & Stock	Catalogue, search by vehicle, restock, low-stock
Customers & Riders	Master data, rider workload and history
Orders	Order creation, stock deduction, fee, invoice
Delivery & OTP	Pack, assign rider, dispatch, OTP-confirmed delivery
Tracking	Public tracking page and WhatsApp link
Reports	Dashboard, top parts, rider performance, 7-day value

5.3 Application Screenshots

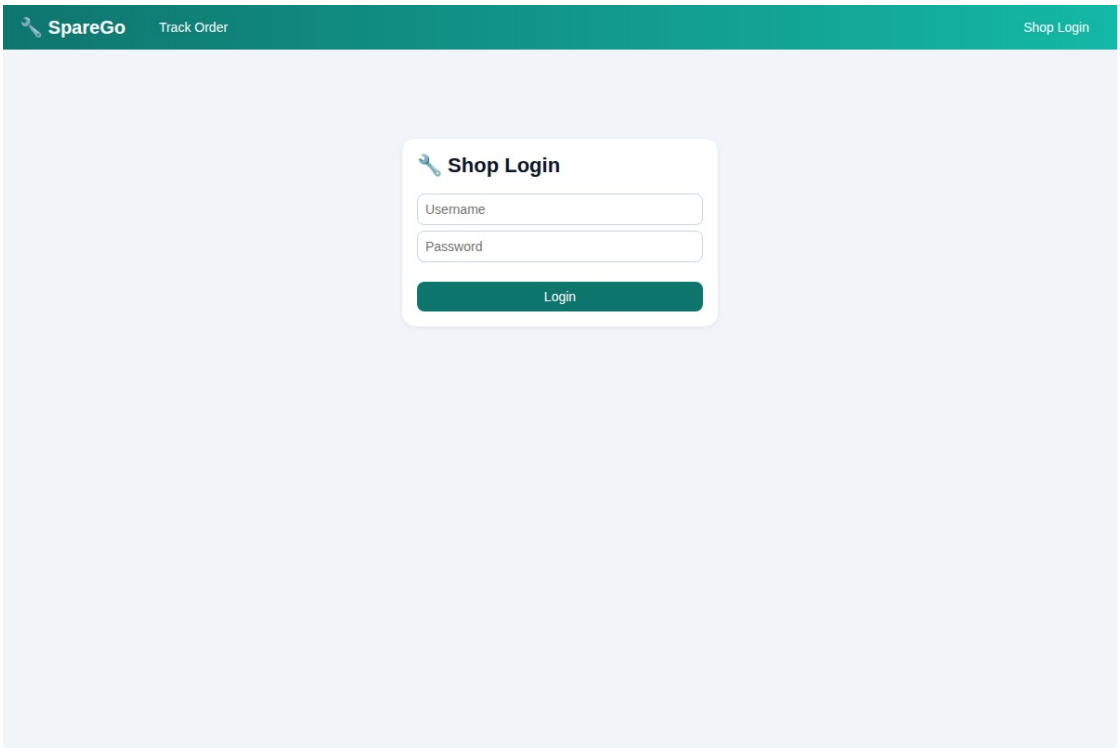


Figure: Login

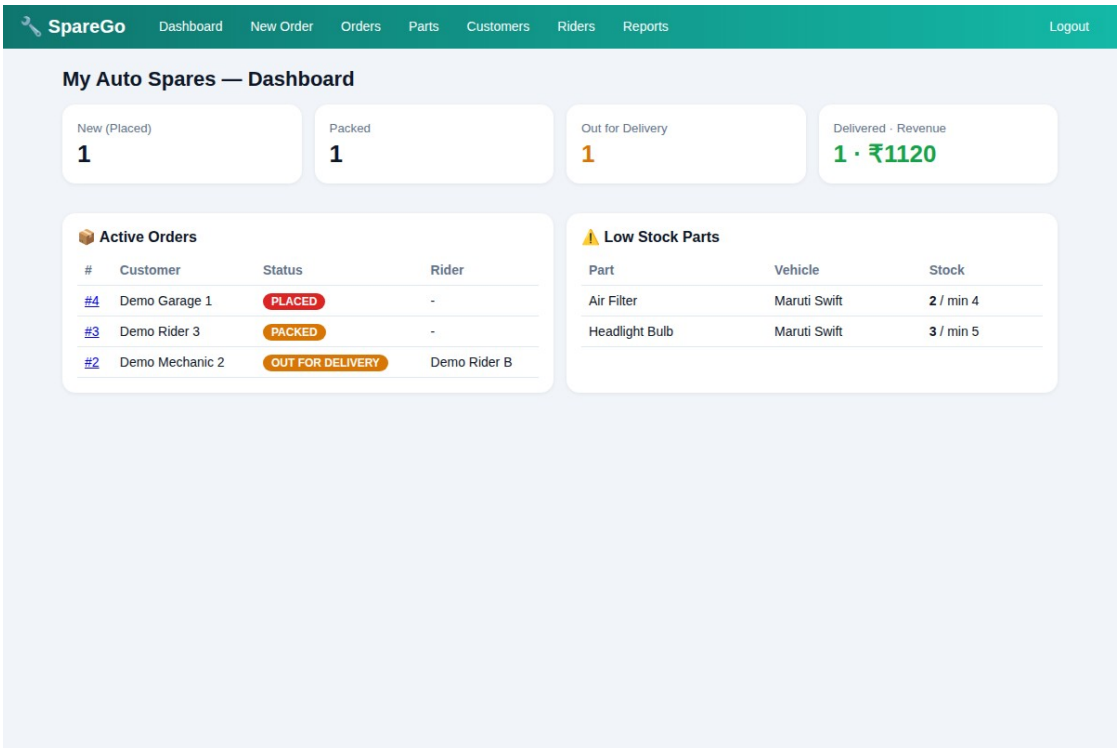


Figure: Dashboard

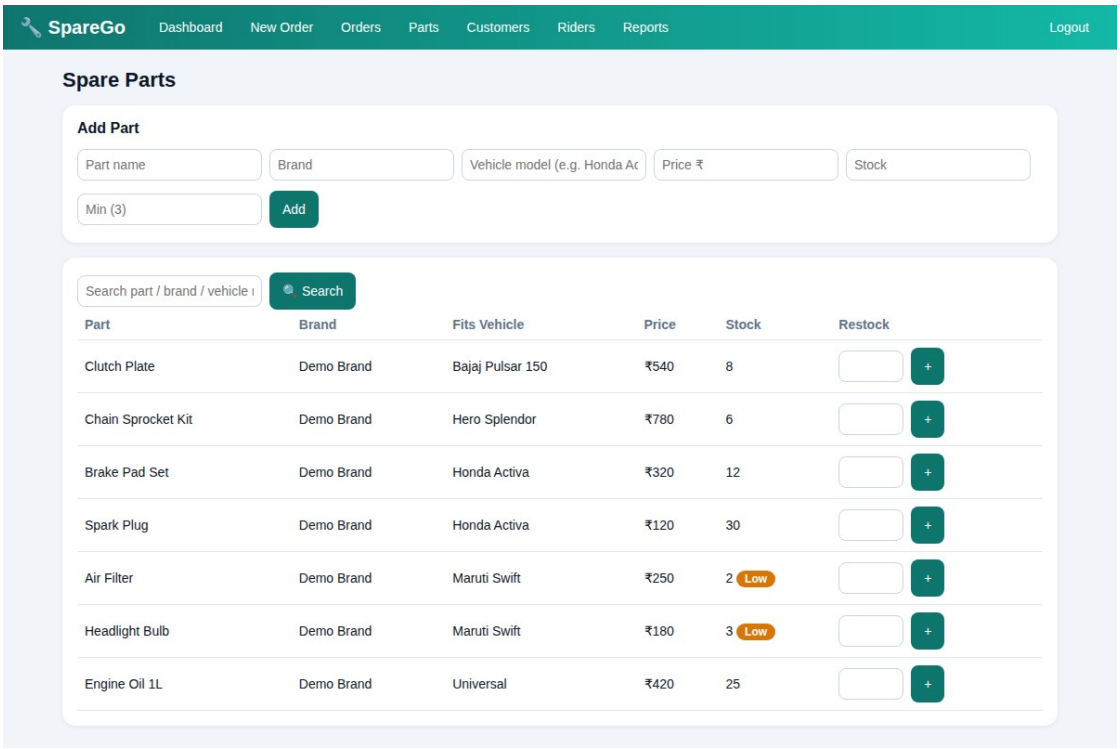


Figure: Parts catalogue and search

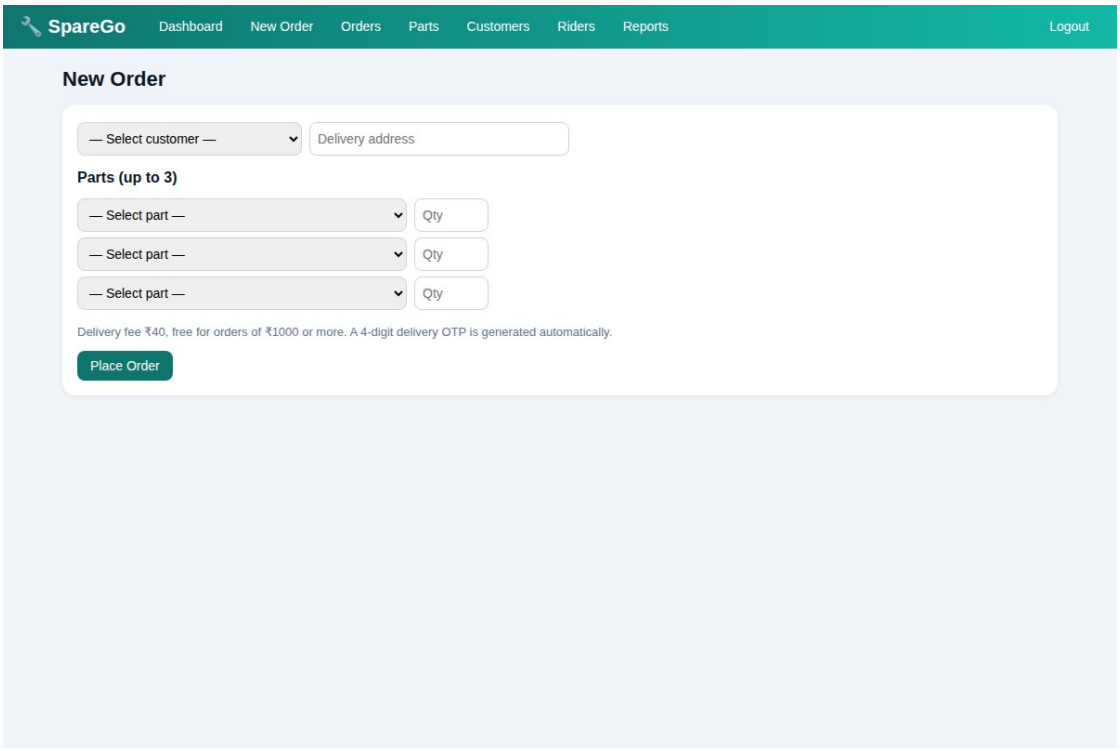


Figure: New order

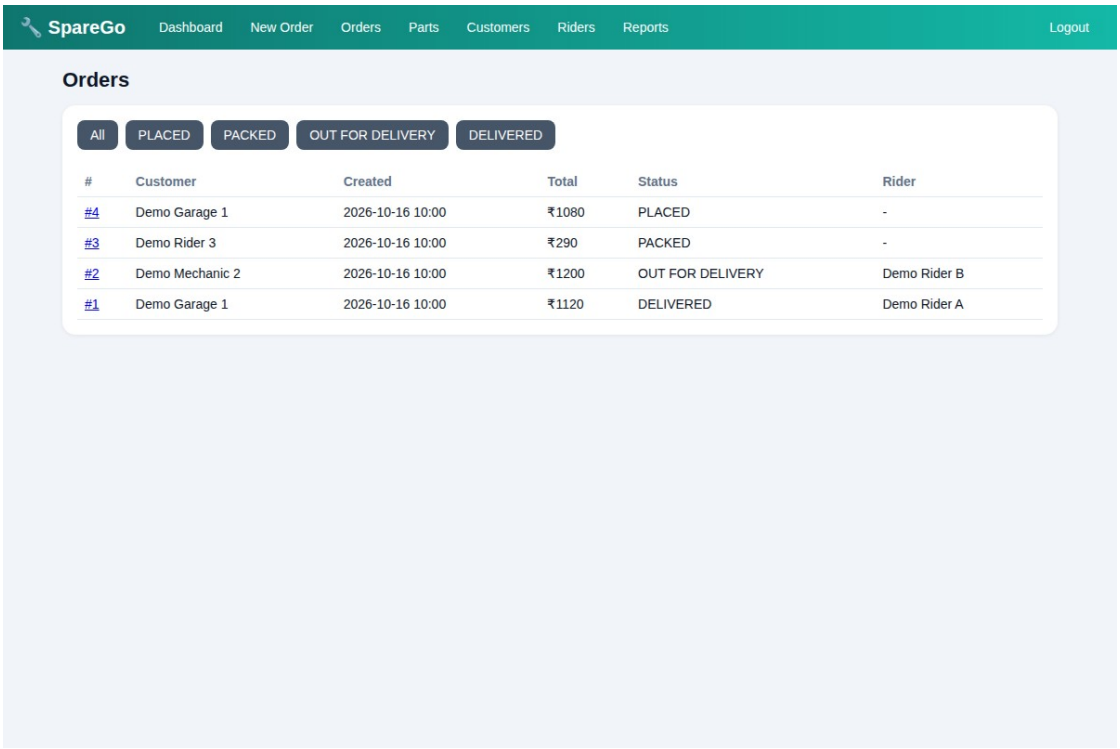


Figure: Order list

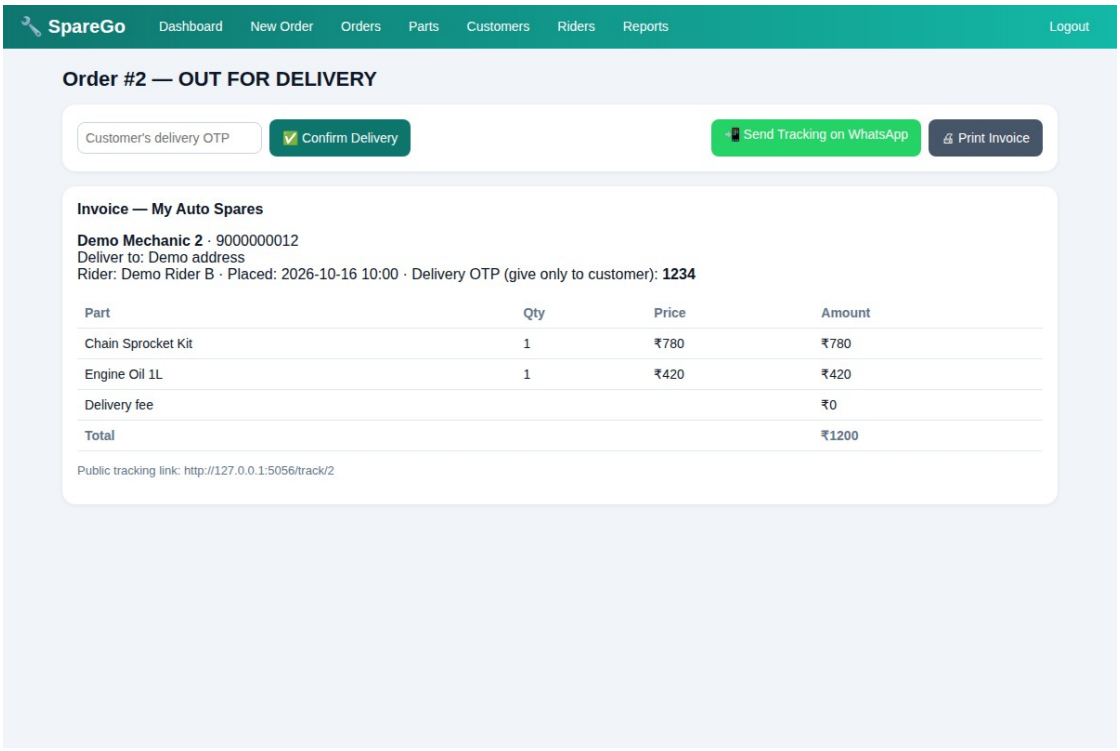


Figure: Order detail, invoice and WhatsApp link

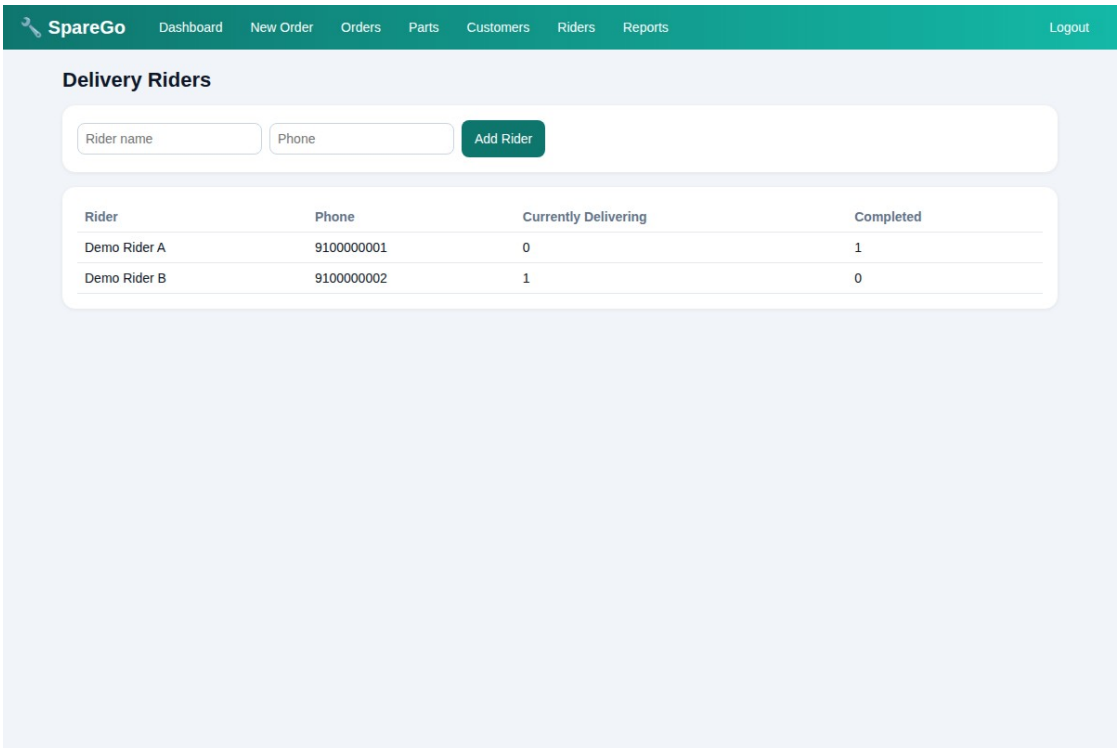


Figure: Riders

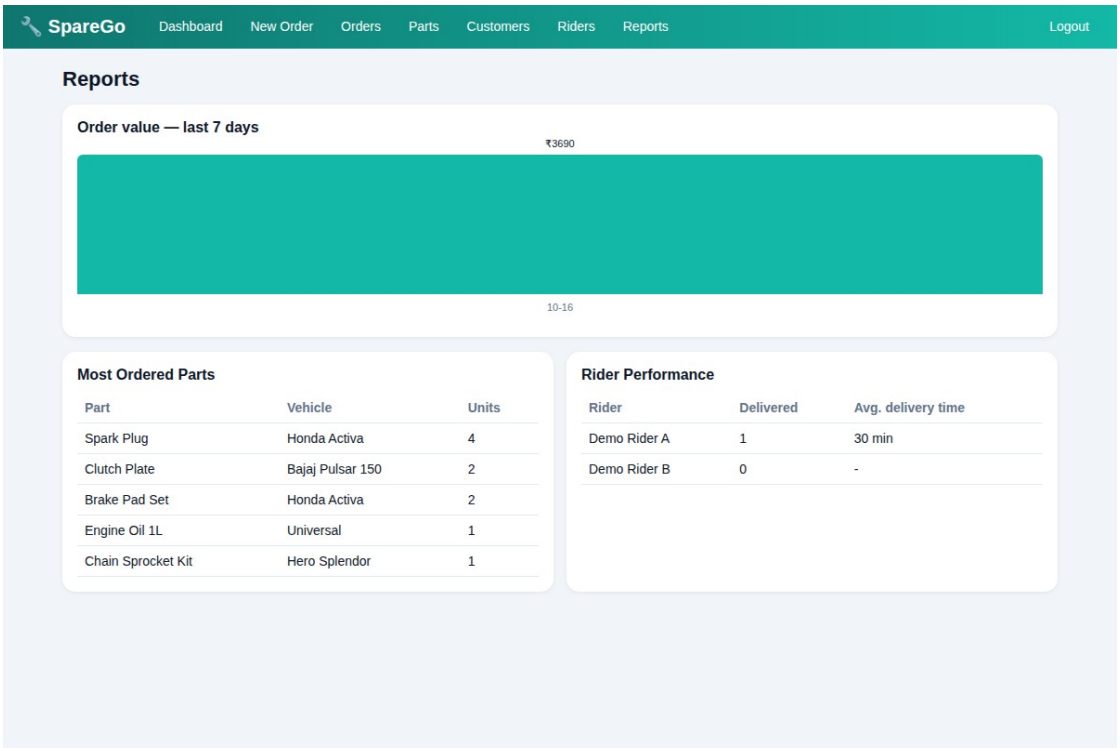


Figure: Reports

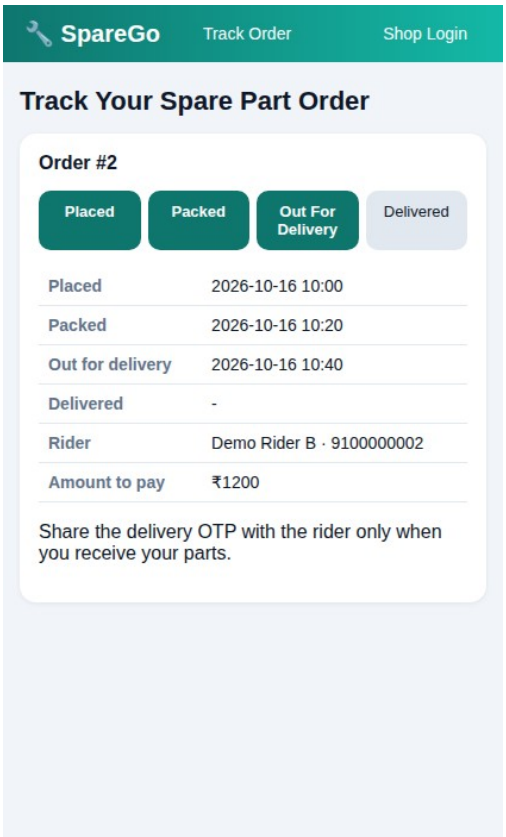


Figure: Customer tracking page (mobile view)

NOTE: Screenshots use demo data. Replace them with real-data screenshots before submission.

Chapter 6: Conclusion and Future Scope

6.1 Conclusion

SpareGo replaces paper and WhatsApp-based ordering with a structured system that finds parts by vehicle model, prevents over-selling, tracks every order and proves delivery with an OTP. All 16 test cases passed. [Add the owner's real feedback after the demo.]

6.2 Existing vs Proposed System

Feature	Existing	SpareGo
Part search	Memory / shelves	Search by part, brand, vehicle
Order record	Notebook / chat	Database with invoice
Status for customer	Phone calls	Tracking link / page
Delivery proof	None	OTP verification
Stock alert	None	Low-stock alerts
Reports	None	Auto reports and rider performance

6.3 Future Scope

- Live GPS tracking of riders and ETA.
- Rider mobile app.
- Online payment / UPI link and COD reconciliation.
- Barcode scanning, supplier purchase orders.
- Multi-shop and staff roles, Hindi/Marathi interface.
- Demand prediction from order history.

Appendix A: Questionnaire

Q1. What type of customer are you? Mechanic / Garage / Individual

Q2. How do you order spare parts now? Visit / Phone / WhatsApp

Q3. How long does it take to know if a part is available?

Q4. Have you received a wrong or incompatible part? Yes / No

Q5. How long does delivery usually take?

Q6. Do you have to call to ask where your order is? Yes / No

Q7. Is delivery ever late enough to affect your work? Yes / No

Q8. Would you like a tracking link on WhatsApp? Yes / No

Q9. Would you prefer to confirm delivery using an OTP? Yes / No

Q10. Would you use a simple app for ordering parts? Yes / No

Appendix B: Manual Test Cases

ID	Test case	Expected	Result
TC01	Wrong password	Error shown	Pass
TC02	Valid login	Dashboard opens	Pass
TC03	Add part	Part listed	Pass
TC04	Invalid phone	Rejected; valid saved	Pass
TC05	Add rider	Rider listed	Pass
TC06	Order qty above stock	Rejected	Pass
TC07	Order 2 x Rs 300	Total 640 (incl. fee 40); stock 10→8	Pass
TC08	Fee rule	Free at 1000+, else 40	Pass
TC09	Mark packed	Status PACKED	Pass
TC10	Dispatch without rider	Blocked	Pass
TC11	Assign rider, dispatch	OUT FOR DELIVERY	Pass
TC12	Wrong OTP	Not delivered	Pass
TC13	Correct OTP	DELIVERED	Pass
TC14	Public tracking	Status visible without login	Pass
TC15	Tracking wrong phone	Not found	Pass
TC16	Admin page without login	Redirect to login	Pass

References

- Flask documentation – https://flask.palletsprojects.com
- SQLite documentation – https://www.sqlite.org/docs.html
- Pressman R., Software Engineering: A Practitioner's Approach.
- Kendall & Kendall, Systems Analysis and Design.